

Questão 2-

a) Existe espera circular? Explique.

Existe um ciclo fechado de dependências entre as três threads e os três recursos. O cenário de travamento ocorria da seguinte forma:

A thread t1 retém r1 e espera por r2.

A thread t2 retém r2 e espera por r3.

A thread t3 retém r3 e espera por r1.

Isso formava o ciclo clássico: $t1 \rightarrow r2 \rightarrow t2 \rightarrow r3 \rightarrow t3 \rightarrow r1 \rightarrow t1$.

b) As quatro condições de impasse estão presentes?

Sim.

Exclusão Mútua: Os mutexes r1, r2 e r3 só permitem um único dono por vez.

Posse e Espera: Cada thread segura um recurso (com o primeiro lock e o sleep) enquanto aguarda o próximo.

Não-Preempção: O sistema operacional não força a liberação de nenhum dos mutexes.

Espera Circular: Conforme explicado no item (a), havia um ciclo completo unindo as três threads.

c) Como prevenir o problema?

A prevenção é feita estabelecendo uma ordem linear/global fixa para a aquisição de recursos. Se definirmos que a ordem correta e obrigatória do sistema é obrigatoriamente crescer do menor índice para o maior ($r1 \rightarrow r2 \rightarrow r3$), nenhuma thread tem a permissão de tentar dar lock em um recurso de menor índice se já possuir um de maior índice.

d) Reescreva o código usando uma política segura de aquisição de recursos.

Código corrigido em `resolucao2.c`